



Rapport de Projet de Programmation Orientée Objet

Implémentation du jeu "Blue Prince"

Jeanne LAKITS
Maxence DEMANGE-ROCHE
Alen GREBENC
M1 - Parcours IPS

Novembre 2025

Responsable de l'UE : Louis ANNABI

Table des matières

1	Introduction	2
2	Architecture Logique et Choix de Conception	3
2.1	La Classe Jeu : L'Orchestrateur (Contrôleur)	3
2.2	La Composition : Joueur et Inventaire	3
2.3	L'Agrégation : Jeu, Manoir et Joueur	4
2.4	Encapsulation des Mécaniques (Classes Piece et Porte)	4
3	Conception Avancée : Hiérarchie des Objets	5
3.1	Héritage et Spécialisation	5
3.2	Polymorphisme	5
4	Implémentation des Fonctionnalités (Barème IPS)	7
5	Résultats et Démonstration	8
5.1	Interface Utilisateur et États Clés	8
5.1.1	Fonctionnalité du Marteau	8
5.2	Démonstration des Mécanismes	9
5.2.1	Ouverture des Portes Verrouillées	9
5.2.2	Victoire et Défaite	10
6	Conclusion	11

Chapitre 1

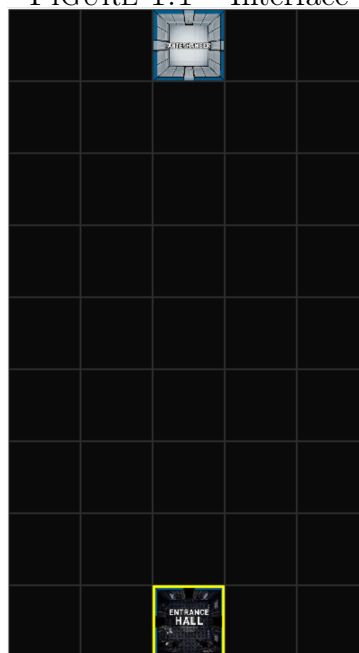
Introduction

Ce document constitue le rapport de projet pour l'unité d'enseignement de Programmation Orientée Objet. L'objectif était de concevoir et d'implémenter une version fonctionnelle du jeu "Blue Prince" en utilisant le langage Python et la bibliothèque Pygame.

Le concept du jeu repose sur l'exploration d'un manoir (une grille 5x9) où le joueur construit son propre chemin en choisissant parmi trois pièces tirées au sort. La progression est limitée par une ressource de "pas" et conditionnée par la gestion d'un inventaire (clés, gemmes, etc.).

En tant qu'étudiants du parcours IPS, ce rapport se concentre sur les exigences spécifiques de notre barème (Table 3) : la justification de nos choix d'implémentation et la démonstration d'une bonne utilisation des classes, attributs et méthodes pour structurer notre code. Nous avons fait des choix architecturaux forts, notamment l'utilisation de la composition et de l'héritage, non pas par obligation de barème, mais parce qu'ils constituent la solution la plus robuste et la plus maintenable.

FIGURE 1.1 – Interface



Chapitre 2

Architecture Logique et Choix de Conception

Notre architecture repose sur une séparation stricte des responsabilités (Single Responsibility Principle). Chaque concept majeur du jeu est encapsulé dans une classe dédiée, dont les interactions sont orchestrées par une classe centrale.

2.1 La Classe Jeu : L'Orchestrateur (Contrôleur)

La classe `Jeu` (`jeu.py`) est le cœur de l'application.

- **Responsabilité** : Elle initialise Pygame, gère la boucle principale d'événements (`en_cours_gestion`), et maintient une machine à états finis simple via l'attribut `self.etat_jeu` (ex : "Déplacement", "Sélection pièces", "Magasin", "Fin"). Elle gère également l'intégralité de la logique d'affichage.
- **Justification du Choix** : Ce choix de conception (proche du patron de conception "Contrôleur") est fondamental. La classe `Jeu` est la seule à interagir directement avec Pygame. Les autres classes (modèles) comme `Manoir` ou `Inventaire` sont agnostiques à la technologie d'affichage. Cela permet un découplage fort : la logique de jeu pure n'est pas polluée par des instructions de dessin.

2.2 La Composition : Joueur et Inventaire

Un des choix de conception majeurs a été de séparer le `Joueur` de son `Inventaire` par **composition**.

- **Description** : La classe `Joueur` (`joueur.py`) est intentionnellement légère. Sa responsabilité unique est de gérer la *position* du joueur sur la grille. Elle *possède* ("has-a") une instance de la classe `Inventaire`.
- **Description** (`Inventaire`) : La classe `Inventaire` (`inventaire.py`) est une classe complexe qui gère *toutes* les ressources. Elle stocke les consommables (pas, pièces d'or, gemmes, clés, dés) et un dictionnaire des objets permanents (Pelle, Kit de Crochetage, etc.) . Elle fournit une API claire pour manipuler ces ressources (ex : `depenser_gemmes`, `depenser_cles`, `ramasser_objet`).

- **Justification du Choix** : Cette séparation évite la création d'une "God Class" `Joueur` qui gèrerait à la fois sa position, son inventaire, et potentiellement d'autres logiques. En utilisant la composition, la classe `Joueur` délègue toute la gestion des ressources à son composant `Inventaire`. C'est une structuration du code plus propre et plus facile à maintenir.

2.3 L'Agrégation : Jeu, Manoir et Joueur

L'état global du jeu est géré par agrégation au sein de la classe `Jeu`.

- **Description** : La classe `Jeu` crée et *possède* les instances de `Manoir` et `Joueur`.
- **Description (Manoir)** : La classe `Manoir` (`manoir.py`) représente le modèle de données de l'environnement. Sa responsabilité est la gestion de la grille 2D (`self.grille`) et de la pioche de pièces (`self.pioche`), chargée depuis `Catalogue_pieces.py`. Elle expose des méthodes claires comme `get_piece_at` ou `tirer_trois_pieces`.
- **Justification du Choix** : Cette structure permet à l'orchestrateur `Jeu` de faire interagir les différents modèles. Par exemple, lors d'un déplacement, `Jeu` demande au `Joueur` sa position, puis au `Manoir` la pièce à cette position, puis initie une interaction entre les deux.

2.4 Encapsulation des Mécaniques (Classes `Piece` et `Porte`)

Nous avons fait le choix de créer des classes distinctes pour les pièces et leurs connexions afin d'encapsuler leur logique métier respective.

- **Description (Piece)** : La classe `Piece` (`ClassePiece.py`) sert de conteneur de données pour une salle, chargeant ses informations (nom, couleur, image, coût, rareté, loot) depuis le catalogue. Elle stocke également un dictionnaire `portes_objets` qui contient les instances réelles de `ClassePorte`.
- **Description (Porte)** : C'est un choix d'implémentation central. Plutôt qu'un simple booléen dans la classe `Piece`, une porte est un objet complexe (`ClassePorte.py`). Elle gère son propre état (`ouverte`) et son niveau de verrouillage (`niveau`).
- **Justification du Choix** : La méthode `Porte.ouvrir(self, joueur)` encapsule toute la logique de vérification requise. C'est la porte elle-même qui vérifie l'inventaire du joueur pour voir s'il possède une clé ou le "Kit de Crochetage" pour une porte de niveau 1. En déplaçant cette logique dans la classe `Porte`, nous simplifions radicalement la classe `Jeu`, qui n'a plus qu'à appeler `porte_cible.ouvrir(joueur)`.

Chapitre 3

Conception Avancée : Hiérarchie des Objets

Bien que l'héritage ne soit pas une exigence pour le parcours IPS, nous avons choisi d'implémenter une hiérarchie de classes complète pour les objets (`ClasseObjet.py`). Ce choix de conception est, selon nous, essentiel pour un code extensible et maintenable.

3.1 Héritage et Spécialisation

Notre conception repose sur une classe de base `Objet` qui définit l'interface commune (nom, description, rareté).

1. **Niveau 1 (Comportement)** : `Objet` a deux enfants : `ObjetConsommable` et `ObjetPermanent`. Ce premier niveau sépare les objets selon leur comportement fondamental (sont-ils consommés?).
2. **Niveau 2 (Spécialisation)** : La hiérarchie des consommables est affinée. `ObjetConsommable` est la classe parente de :
 - **Nourriture** : Classe de base pour les objets redonnant des pas (Pomme, Banane, etc.).
 - **Ressource** : Classe de base pour les "monnaies" (Clé, Gemme, Pièce d'Or, Dé).
3. **Niveau 3 (Implémentation)** : Les classes concrètes (`Pomme`, `Cle`, `Pelle`) implémentent le comportement final.

3.2 Polymorphisme

Cette structure d'héritage trouve sa justification dans le mécanisme puissant suivant : le polymorphisme.

- **Le Polymorphisme en Action** : L'avantage de l'héritage est visible dans la méthode `Inventaire.ramasser_objet`.

```

def ramasser_objet(self, objet_instance, joueur_instance):
    """
    Ramasse un objet et applique son effet.

    Args:
        objet_instance: Instance de l'objet à ramasser (doit avoir les attributs
            `type_objet`, `nom`, et la méthode `utiliser`).
        joueur_instance: Instance du joueur recevant l'objet.
    """
    if objet_instance.type_objet == "Permanent":
        if objet_instance.nom in self.objets_permanents and not self.objets_permanents[objet_instance.nom]:
            self.objets_permanents[objet_instance.nom] = True
            objet_instance.utiliser(joueur_instance) #Appel polymorphique
            print(f"Objet permanent obtenu et activé : {objet_instance.nom}")
        elif objet_instance.nom in self.objets_permanents and self.objets_permanents[objet_instance.nom]:
            print(f"Objet permanent ignoré (déjà possédé) : {objet_instance.nom}")
        else:
            print(f"Avertissement : Objet permanent '{objet_instance.nom}' non pré")

    elif objet_instance.type_objet == "Consommable":
        objet_instance.utiliser(joueur_instance) #Appel polymorphique

    else:
        print(f"Erreur: Type d'objet inconnu : {objet_instance.type_objet}")

```

FIGURE 3.1 – Extrait de inventaire.py - Polymorphisme

Justification du Choix : L'appel à `objet_instance.utiliser(joueur_instance)` est polymorphique. L'`Inventaire` ne sait pas, et n'a pas besoin de savoir, si l'objet est une `Pomme` ou une `Cle`. Il appelle la méthode `utiliser`, et c'est l'implémentation spécifique de la classe de l'objet (ex : `Pomme.utiliser` ou `Cle.utiliser`) qui exécute la bonne action. Cette architecture rend l'ajout de nouveaux objets trivial, sans avoir à modifier la classe `Inventaire`.

Chapitre 4

Implémentation des Fonctionnalités (Barème IPS)

Cette architecture nous a permis d'implémenter l'ensemble des fonctionnalités requises par le barème IPS (Table 1 de l'énoncé).

- **Interface et Déplacement** : Gérés par `Jeu` (pour l'affichage et les inputs ZQSD) et `ClassePorte` (pour la logique d'ouverture). La perte de pas est gérée par `Joueur.deplacer_vers` qui décrémente `self.inventaire.pas`.
- **Tirage et Choix des Pièces** : Gérés par `Manoir.tirer_trois_pieces` (qui implémente la logique de rareté et de pièce gratuite) et par `Jeu` (état "Selection pieces").
- **Gestion des Ressources (Pas, Gemmes, Clés, Dés)** : Entièrement gérée par la classe `Inventaire` et ses méthodes `depenser_cles`, `depenser_gemmes`, `depenser_des`, etc.
- **Logique Aléatoire** : L'aléatoire est encapsulé dans les classes responsables :
 - `Porte.generer_niveau_verrouillage` pour les portes.
 - `Manoir.tirer_trois_pieces` pour le tirage.
 - `Jeu._generer_et_ramasser_butin` pour le loot.
- **Objets Permanents** : Le "Kit de Crochetage", le "Déecteur de Métaux" et la "Patte de Lapin" sont implémentés via `ClasseObjet.py` et leur possession est vérifiée par `Inventaire.a_objet_permanent`. De plus, bien que non requis pour le barème IPS, nous avons essayé de nous pencher sur des aspects de la Table 2 (hors IPS), mais on ne peut garantir totalement leur fonctionnalité à exception de **Marteau** que nous allons voir dans la prochaine section.

Chapitre 5

Résultats et Démonstration

Cette section présente les résultats d'exécution du jeu, prouvant la fonctionnalité des mécanismes clés implémentés grâce à l'architecture Orientée Objet.

5.1 Interface Utilisateur et États Clés

L'interface de jeu (`Jeu.affichage_grille`) sépare clairement la grille du manoir et l'inventaire du joueur. La gestion dynamique des états permet de fournir des instructions spécifiques au contexte.

FIGURE 5.1 – Interface de jeu

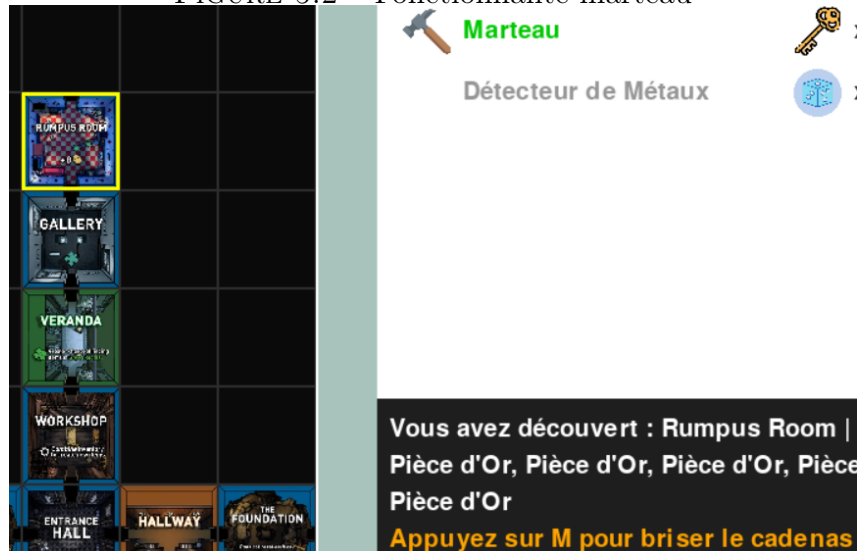


5.1.1 Fonctionnalité du Marteau

Le Marteau (`Marteau`) permet d'ouvrir un coffre sans dépenser de clé. Cette interaction est rendue possible par :

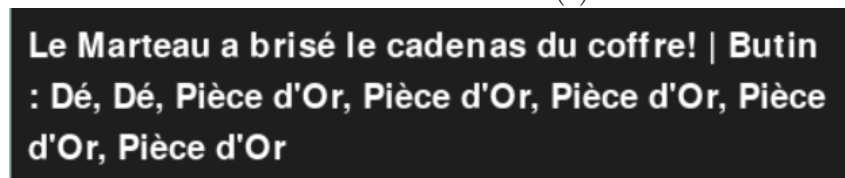
- La vérification d'état dans `Jeu.affichage_message_statut` qui affiche l'instruction si la pièce actuelle (`piece_actuelle.a_coffre`) possède un coffre qui est verrouillé.

FIGURE 5.2 – Fonctionnalité marteau



- La méthode `Piece.tenter_casser_cadenas` qui change l'état du cadenas (`cadenas_casse = True`) et retourne le butin.

FIGURE 5.3 – Butin(s)



- La méthode `Jeu.utiliser_objet_permanent("Marteau")` qui orchestre l'interaction et le ramassage du butin.

5.2 Démonstration des Mécanismes

5.2.1 Ouverture des Portes Verrouillées

L'ouverture des portes verrouillées (Niveau 1 ou 2) nécessite l'état `Deblocage Porte`, prouvant la bonne encapsulation de la logique de dépense :

1. Le joueur tente un mouvement vers une porte verrouillée.
2. La classe `Jeu` détecte le coût et passe à l'état `Deblocage Porte`.
3. La méthode `Jeu.tenter_deblocage` est appelée, vérifiant si le joueur utilise une Clé ou le Kit de Crochetage.

Ceci confirme que les **Clés** et le **Kit de Crochetage** gèrent l'accès, séparément du coût en **Gemmes** (pour l'achat d'une nouvelle pièce).

FIGURE 5.4 – Exemple de déblocage car porte verrouillée



5.2.2 Victoire et Défaite

Les conditions de fin de jeu sont gérées par `Jeu.verifcation_fin_jeu` :

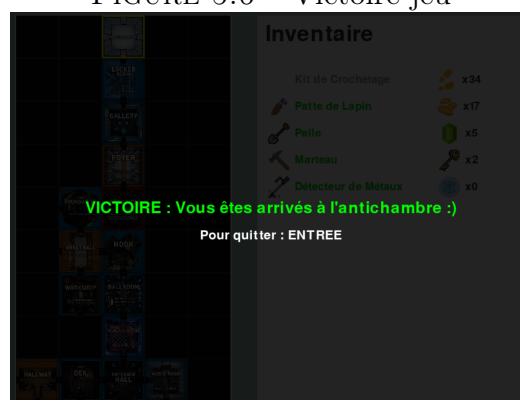
- **Défaite** : Atteinte lorsque le nombre de pas du joueur devient nul.

FIGURE 5.5 – Défaite jeu



- **Victoire** : Atteinte lorsque le joueur entre dans la pièce Antechamber.

FIGURE 5.6 – Victoire jeu



Chapitre 6

Conclusion

Ce projet a été une application concrète des principes de Programmation Orientée Objet pour résoudre un problème complexe. L'architecture mise en place, basée sur une séparation stricte des responsabilités (via les classes `Jeu`, `Manoir`, `Joueur`, `Inventaire`) et l'utilisation de patrons de conception avancés (Composition, Héritage, Polymorphisme, Factory), s'est avérée robuste et modulable.

Ces choix d'implémentation, bien qu'allant au-delà des exigences minimales du parcours IPS, ont été essentiels pour produire un code propre et fonctionnel. Notre implémentation répond à l'intégralité des fonctionnalités du barème principal (Table 1) et démontre une structuration du code mûrement réfléchie.